

GLOBAL ACADEMY OF TECHNOLOGY

growing ahead of time

Autonomous Institute, Affiliated to VTU

Department of Artificial Intelligence & Machine Learning

AML23702

Advance Natural Language Processing

AML23702_Syllabus.pdf

VASUGI I

ASSISTANT PROFESSOR

DEPARTMENT OF AI & ML

GLOBAL ACADEMY
OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

Module-4
Topic Modeling

Topic Modeling : Mathematical foundations for LDA : Multinomial and Dirichlet distributions, Intuition behind LDA, LDA Generative model, Latent Dirichlet Allocation Algorithm and Implementation, Gibbs Sampling

GLOBAL ACADEMY OF TECHNOLOGY
growing ahead of time
Autonomous Institute, Affiliated to VTU

Pre-Session Reading

- Basic understanding of LDA - https://www.ibm.com/think/topics/latent-dirichlet-allocation
- Beginner's Guide to LDA - https://medium.com/data-science/latent-dirichlet-allocation-lda-9d1cd064ffa2
- Latent Dirichlet allocation (LDA) and topic modeling: models, applications, a survey - https://dl.acm.org/doi/10.1007/s11042-018-6894-4
- https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf

Topic Modeling

with Latent Dirichlet Allocation

From the intuition of “documents as mixtures of topics” to Dirichlet mathematics, the LDA generative model, collapsed Gibbs sampling and a working implementation.



What we will cover

Five parts — building from motivation to a running LDA sampler

1

Foundations & applications

What topic modeling is, why it exists, and where it runs in production today.

2

Mathematical foundations

Categorical, Multinomial, Beta, Dirichlet and conjugacy — with worked numbers.

3

Intuition behind LDA

The generative story, plate notation, and what the model actually assumes.

4

LDA algorithm & Gibbs sampling

The collapsed Gibbs update derived, traced by hand, then coded.

Thread running through the deck: one tiny corpus of documents is preprocessed, modelled, sampled and evaluated — so every formula is tied to numbers you can check by hand.

Assumed background

Basic probability (random variables, conditional probability, Bayes' rule) · elementary calculus · Python. No prior exposure to Bayesian inference or MCMC is required — both are developed from first principles.

Foundations & Applications

What topic modeling is, the problem it solves, and the systems that depend on it every day.



Text streams from many domains flowing into one analytical model

What is topic modeling?

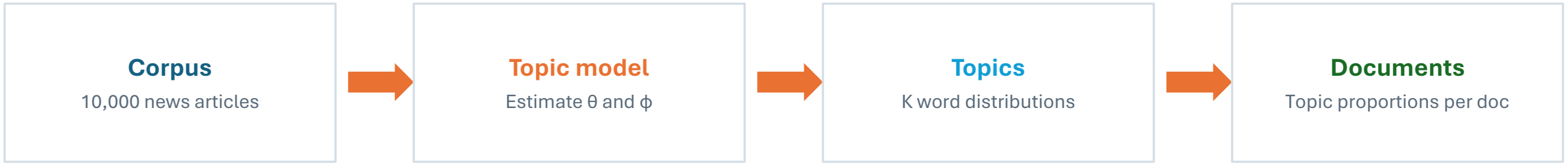
An unsupervised method for discovering the latent themes that generate a document collection

Working definition

Topic modeling discovers a small set of recurring word-usage patterns — topics — from a large unlabelled corpus, and describes every document as a proportion of those topics.

No labels, no taxonomy, no training data. The structure is inferred from word co-occurrence alone.

- **Latent.** Topics are never observed — only words are.
- **Probabilistic.** A topic is a distribution over the vocabulary; a document is a distribution over topics.
- **Soft clustering.** A document belongs to many topics at once, unlike k-means.
- **Bag of words.** Word order is discarded; co-occurrence carries the signal.



Key shift in thinking: instead of asking “which class is this document?” we ask “in what proportions was this document generated?”

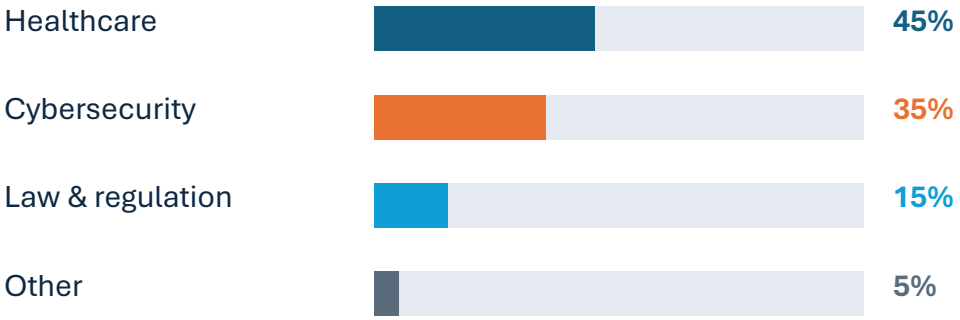
The core idea: documents are mixtures of topics

One article about a hospital data breach is genuinely part health, part technology, part law

Document d_7

“The hospital confirmed that attackers accessed patient records for six weeks before the breach was detected. Regulators have opened an inquiry under privacy law.”

Inferred topic proportions θ_7



Each topic is itself a distribution over words

Topic 4 — Healthcare

`patient 0.061 · hospital 0.048 · clinical 0.033 · care 0.029 · record 0.021`

Topic 11 — Cybersecurity

`breach 0.057 · attacker 0.044 · access 0.038 · detect 0.026 · server 0.019`

Topic 2 — Law & regulation

`regulator 0.052 · privacy 0.041 · inquiry 0.030 · law 0.028 · fine 0.022`

Two levels of mixture: documents mix topics (θ), and topics mix words (ϕ). LDA estimates both from the raw text at once.

What a topic actually is

A ranked probability distribution over the whole vocabulary — the label is ours, not the model's

Rank	Topic 4 (ϕ_4)	p(w|z=4)	Topic 11 (ϕ_{11})	p(w|z=11)
1	patient	0.061	breach	0.057
2	hospital	0.048	attacker	0.044
3	clinical	0.033	access	0.038
4	care	0.029	detect	0.026
5	record	0.021	server	0.019
…	… (remaining 49,995 words)	→ 0	… (remaining 49,995 words)	→ 0
	Sum over vocabulary	1.000	Sum over vocabulary	1.000

The model outputs numbers, not names

"Healthcare" is a human reading of the top-ranked words. Nothing in LDA names a topic — always inspect the top words before trusting a label.

Every word has non-zero mass

Dirichlet smoothing keeps p(w|z) > 0 for all words, so an unseen word never makes a document impossible.

Topics are exchangeable

Topic indices carry no meaning and can permute between runs — compare runs by word content, not index.

Why topic modeling?

Four jobs it does that supervised classification and keyword search cannot

1

Organise the unlabelled

Labels for 10M documents are unaffordable. Topic models need none — structure comes from co-occurrence.

2

Reduce dimensionality

A 50,000-word sparse vector becomes a dense 50-dimensional θ that works as a feature for downstream models.

3

Discover the unexpected

Emerging themes appear without being asked for — a new defect, a new complaint, a new narrative.

4

Explain, don't just predict

Each decision traces to specific words with explicit probabilities — unlike a dense neural embedding.

Compared with the alternatives

- Keyword search — finds documents you already know to ask for.
- Supervised classifier — can only return categories you defined in advance.
- k-means on TF-IDF — forces each document into exactly one cluster.
- **Topic model — soft, probabilistic, interpretable, and label-free.**

When it is the right tool

Thousands of documents or more, each at least a paragraph long, where you need an overview or a feature — not a precise per-document label.

When it is not: short texts (tweets, queries), tiny corpora, or tasks where a labelled classifier already performs well.

Where topic modeling is used

Six domains where latent-theme discovery is standard practice

News & media

Section tagging, story clustering, coverage analysis across wires and archives.

Customer voice

Reviews, NPS comments and support tickets grouped into recurring complaint themes.

Healthcare

Clinical notes and discharge summaries mined for comorbidity and care-pathway patterns.

Legal & compliance

E-discovery over millions of documents; contract-clause and policy-theme grouping.

Science & patents

Mapping research fronts, funding trends and technology landscapes over time.

Security & operations

Incident reports, log narratives and threat intel bucketed into failure themes.

Common shape of the problem: a large, growing, unlabelled text stream that somebody must summarise, route or monitor — and where the categories themselves keep changing.

Real-time applications in production

Where a topic model sits inside a live system rather than an offline report

System	What is modelled	Latency path	Decision it drives
News recommendation	Article topic vector θ at ingest	Model offline, inference per article (~ms)	Cold-start article → reader matching
Support ticket triage	Incoming ticket text	Streaming inference on arrival	Route to queue, detect a spike in a theme
Review monitoring	Product reviews per SKU	Micro-batch, hourly	Alert when a defect topic rises sharply
Content moderation	Post / comment text	Inference in the publish path	Prioritise reviewer attention by theme
Ad & feed relevance	User history as a topic profile	Profile refreshed continuously	Blend topical interest into ranking
Ops & incident intel	Alert and log narratives	Rolling window over the stream	Cluster a storm of alerts into one incident

The practical pattern: train LDA in batch on a large historical corpus, then run only the cheap per-document inference step online.

Topics drift. Production systems retrain on a schedule and align new topics to old ones so dashboards and alerts stay comparable.

An end-to-end topic modeling pipeline

Six stages — most of the real effort sits in the first two

1 Collect

Crawl, export or stream raw documents into one store.

2 Preprocess

Tokenise, lowercase, remove stopwords, lemmatise, build n-grams.

3 Vectorise

Build the vocabulary and the document-term count matrix.

4 Fit LDA

Choose K and priors; run Gibbs sampling or variational inference.

5 Evaluate

Coherence, perplexity, and human inspection of top words.

6 Serve

Infer θ for new documents; feed search, routing or dashboards.

Where the time actually goes

Preprocessing decides the topics you get. Aggressive stopword lists, lemmatisation and bigram detection change the result far more than tuning α or β.

Rule of thumb: prune words appearing in fewer than 5 documents or in more than 50% of them.

Typical production shape

Nightly batch training on Spark or a single large machine; the fitted ϕ matrix shipped as an artefact; a lightweight service that folds in new documents and returns θ in milliseconds.

LDA among the alternatives

Why the Bayesian generative formulation won for interpretable topics

Method	Basis	Output	Strength	Main limitation
LSA / LSI	SVD on TF-IDF	Dense factors	Fast, deterministic	Factors can be negative — not interpretable
pLSA	Max-likelihood mixture	p(z|d), p(w|z)	First true topic model	No prior on θ → overfits, cannot score new docs
NMF	Non-negative factorisation	Parts-based factors	Often crisp on short text	Not probabilistic; no generative story
LDA	Bayesian generative model	θ and ϕ with priors	Principled, generalises to new documents	Bag of words; K must be chosen
BERTopic	Embeddings + clustering	Hard clusters + c-TF-IDF	Captures context and word order	Heavy, hard clusters, less transparent

pLSA → LDA is exactly the step of placing a Dirichlet prior over the document-topic proportions. That single change gives regularisation, a way to score unseen documents, and the whole Bayesian machinery we develop in Part 2.

From raw text to a document-term matrix

The toy corpus used for every worked example in this deck

Raw corpus

d_1: "The patient visited the hospital for a scan."

d_2: "The hospital upgraded its server after a breach."

d_3: "An attacker caused a data breach on the server."

1. Lowercase + tokenise the / patient / visited / the / hospital / …

2. Remove stopwords drops the, a, for, an, its, on, after

3. Lemmatise visited → visit, caused → cause

4. Prune & build vocabulary V = {patient, hospital, scan, server, breach, attacker, data, …}

Document-term count matrix

	patient	hospital	scan	server	breach	attacker	data
d_1	1	1	1	0	0	0	0
d_2	0	1	0	1	1	0	0
d_3	0	0	0	1	1	1	1

This matrix is the entire input to LDA. Word order inside each document is gone — only which words appeared, and how often, survives.

Ten words, three documents, two intended themes (care vs. security) — small enough to sample by hand in Part 4.

Mathematical Foundations

Categorical and Multinomial distributions, conjugacy, and the Dirichlet — each developed with worked numbers.

Generative models and the notation we will use

A generative model is a recipe for producing data; inference runs that recipe backwards

The generative viewpoint

Forward: parameters → data. Write down a probabilistic story that could have produced the corpus.

Backward (inference): data → parameters. Given the documents we actually observed, which hidden settings most plausibly produced them?

$$\theta_{\partial} \in \Delta^{k-1} : \ \theta_{\partial k} \geq 0, \ \Sigma_k \theta_{\partial k} = 1$$

Every θ and ϕ lives on a simplex — a vector of non-negative numbers summing to one.

Only w is observed. Everything else — z, θ, ϕ — must be inferred, which is why LDA needs Bayesian machinery rather than a closed-form formula.

Symbol	Meaning	Size
D	Number of documents	scalar
V	Vocabulary size	scalar
K	Number of topics (chosen by us)	scalar
N_{∂}	Number of word tokens in document d	scalar
$w_{\partial n}$	The n-th observed word token in d	observed
$z_{\partial n}$	Topic assignment of that token	hidden
θ_{∂}	Topic proportions for document d	K-vector
ϕ_k	Word distribution of topic k	V-vector
α, β	Dirichlet hyperparameters (priors)	scalars

From Bernoulli to Categorical to Multinomial

The distributions that describe drawing words from a topic

Bernoulli

One trial, two outcomes.

Categorical

One trial, V outcomes. One word drawn from the vocabulary.

Multinomial

N independent trials, V outcomes. Counts of each word in a document.

$$p(x_1,\ldots,x_V \mid N, \varphi) = \left(N! \,/\, (x_1! \cdots x_V!) \right) \cdot \varphi_1^{x_1} \cdots \varphi_V^{x_V}$$

$$\text{with } \Sigma_v\, x_v = N \;\text{ and }\; \Sigma_v\, \varphi_v = 1$$

The multinomial probability mass function — the likelihood of a bag of words given a topic's word distribution.

Why this matters for LDA

Each word token is a Categorical draw from ϕ_k. A whole document, as a bag of words, is Multinomial.

Properties worth remembering

$E[x_v] = N\varphi_v$ · $\mathrm{Var}[x_v] = N\varphi_v(1-\varphi_v)$ · $\mathrm{Cov}[x_i, x_j] = -N\varphi_i\varphi_j$ — counts are negatively correlated because they must sum to N.

Worked example — the Multinomial likelihood

A 10-word document drawn from a single topic

Setup

Vocabulary of three words: {patient, hospital, breach}.

Topic 1 word distribution: $\varphi_1 = (0.5, 0.3, 0.2)$.

A document of N = 10 tokens is observed with counts

x = (patient 5, hospital 3, breach 2).

How likely is this document under topic 1?

$$p(x \mid \varphi_1) = (10! / (5!\cdot 3!\cdot 2!)) \cdot 0.5^5 \cdot 0.3^3 \cdot 0.2^2$$

$$= 2520 \cdot 0.03125 \cdot 0.027 \cdot 0.04$$

$$= 0.08505$$

The multinomial coefficient 10!/(5!·3!·2!) = 2520 counts the orderings of the same bag.

Comparing two topics for the same document

Topic	ϕ = (patient, hospital, breach)	Likelihood of x = (5, 3, 2)	Relative support
Topic 1 — care	(0.50, 0.30, 0.20)	2520 · 0.03125 · 0.027 · 0.04 = 0.08505	≈ 31× more likely
Topic 2 — security	(0.10, 0.20, 0.70)	2520 · 0.00001 · 0.008 · 0.49 = 0.00099	baseline

This ratio is exactly the signal LDA exploits: the same bag of words is far more probable under one topic than another, and that drives every sampling decision in Part 4.

Conjugacy — Beta and Binomial

Pick a prior whose shape survives the update, and the posterior stays in closed form

$$\text{posterior} \propto \text{likelihood} \times \text{prior} \qquad p(\theta \mid x) \propto p(x \mid \theta) \cdot p(\theta)$$

Conjugate prior: a prior family such that the posterior belongs to the same family.

Worked example — Beta–Binomial

Prior belief about a coin: $\theta \sim \text{Beta}(2, 2)$ — symmetric, mean 0.5, mildly informative.

Observe 8 heads in 10 flips: x = 8, N − x = 2.

Posterior: $\text{Beta}(2 + 8, 2 + 2) = \text{Beta}(10, 4)$.

Posterior mean = 10 / (10 + 4) = 0.714 — pulled below the raw 0.8 estimate by the prior.

The pattern to carry forward

Updating is just adding counts to the prior parameters.

$$\text{Beta}(a, b) + (x \text{ heads}, y \text{ tails}) \rightarrow \text{Beta}(a + x, b + y)$$

The Dirichlet is the Beta generalised from 2 outcomes to K. Exactly the same 'prior + counts' rule will give us the LDA Gibbs update.

Read a, b as pseudo-counts: Beta(2,2) behaves like having already seen one head and one tail before any data arrives.

The Dirichlet distribution

A distribution over distributions — the natural prior for θ and ϕ

$$p(\theta \mid \alpha) = \left(\Gamma(\Sigma_k \alpha_k) \,/\, \Pi_k \Gamma(\alpha_k) \right) \cdot \Pi_k \theta_k^{(\alpha_k - 1)}$$

$$\text{support: } \theta_k \geq 0,\ \Sigma_k \theta_k = 1$$

A draw from a Dirichlet is itself a probability vector — one point on the simplex.

Moments

$$E[\theta_k] = \alpha_k / \alpha_0, \text{ where } \alpha_0 = \Sigma_k \alpha_k$$

$$\mathrm{Var}[\theta_k] = \alpha_k(\alpha_0 - \alpha_k) / (\alpha_0^2(\alpha_0 + 1))$$

α_0 is the concentration: larger α_0 → draws cluster tightly around the mean.

Simplex-valued

Every sample is a valid probability vector, so it can be used directly as θ_d or ϕ_k.

Conjugate to Multinomial

Dirichlet(α) + counts n → Dirichlet(α + n). Updating is again just adding counts.

Symmetric by default

LDA usually sets all α_k equal, so a single scalar controls sparsity.

Generalises the Beta

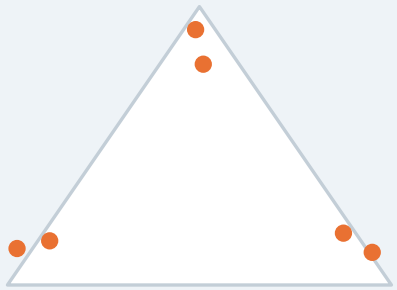
Dirichlet with K = 2 is exactly the Beta distribution.

In LDA: θ_d ~ Dirichlet(α) over topics, and ϕ_k ~ Dirichlet(β) over the vocabulary.

Reading the parameter α

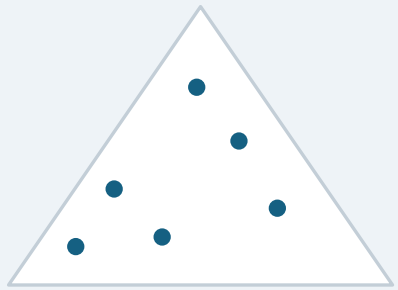
One scalar controls whether documents are focused or diffuse

$\alpha = 0.1$ Sparse



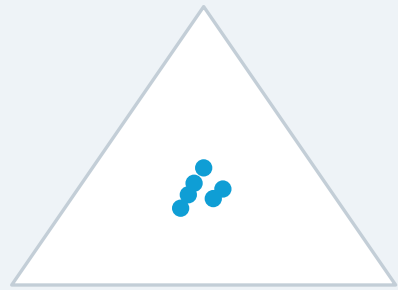
Mass pushed to the corners of the simplex. A document is dominated by one or two topics — the usual choice for real corpora.

$\alpha = 1.0$ Uniform



Every point on the simplex is equally likely. A flat, uninformative prior.

$\alpha = 10$ Dense



Draws cluster near the centre. Every document mixes all topics almost equally — rarely what we want.

Same logic applies to β

β controls topic–word sparsity. Small β (0.01–0.1) gives topics built from a few highly specific words; large β gives broad, vague topics.

Common defaults: α = 50/K and β = 0.01 (Griffiths & Steyvers), or α = β = 1/K in many libraries. Both encourage sparse, readable topics.

Worked example — updating a Dirichlet

Prior pseudo-counts plus observed counts give the posterior directly

Setup

$K = 3$ topics. Symmetric prior $\alpha = (0.5, 0.5, 0.5)$, so $\alpha_0 = 1.5$.

Document d has 20 tokens currently assigned as

$n_d = (\text{topic1: } 12,\ \text{topic2: } 6,\ \text{topic3: } 2)$.

What is the posterior over that document's topic proportions θ_d?

$$p(\theta_d \mid n_d, \alpha) = \text{Dirichlet}(\alpha + n_d)$$

$$= \text{Dirichlet}(12.5,\ 6.5,\ 2.5)$$

Conjugacy at work: no integral, just add the counts to the prior.

Posterior means compared with the raw frequencies

Topic	Raw count n_k	Raw fraction	Posterior mean $(\alpha_k + n_k)/(\alpha_0 + N)$	Effect of the prior
Topic 1	12	12/20 = 0.600	12.5 / 21.5 = 0.581	shrunk slightly downwards
Topic 2	6	6/20 = 0.300	6.5 / 21.5 = 0.302	essentially unchanged
Topic 3	2	2/20 = 0.100	2.5 / 21.5 = 0.116	lifted away from zero

The prior acts as smoothing: no topic is ever assigned probability exactly zero, which is precisely what keeps the Gibbs sampler in Part 4 able to revisit any topic.

Intuition Behind LDA

The generative story a human can retell, then the same story written as a graphical model.

The writer metaphor

How LDA imagines a document was written, one word at a time



1. Before writing

The author picks a blend for this article — say 60% healthcare, 30% security, 10% law. That blend is θ_d, drawn from Dirichlet(α).

2. For every word

Spin the θ_d wheel to choose a topic z; that selects one jar on the desk.

3. Then draw a word

Reach into the chosen jar and pull a word according to that topic's word distribution ϕ_z.

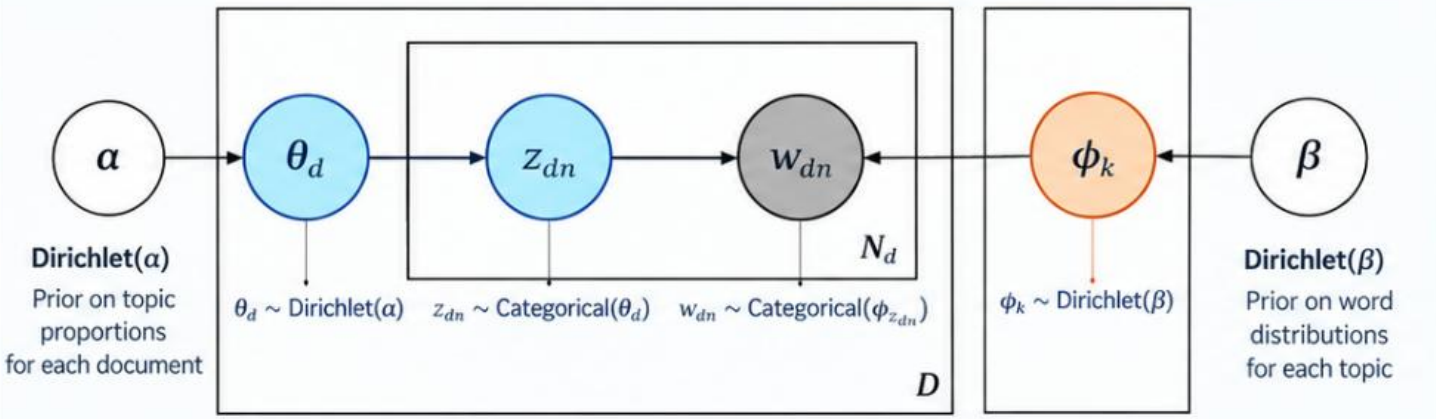
4. Repeat N times

The bag of words that results is the document we actually observe.

Nobody claims writers work this way. The story matters because it defines a probability for every possible corpus — and inverting it recovers topics that are genuinely useful.

Plate notation — the model in one diagram

Plates mean “repeat”; shaded means “observed”



Node	Distribution	Role / Meaning
α	fixed hyperparameter	Prior concentration on document-topic mixtures. Controls sparsity of θ.
θ_d	Dirichlet(α)	**Topic proportions for document** d. A K-dimensional probability vector.
z_{dn}	Categorical(θ_d)	Topic chosen for the n-th word in document d.
w_{dn}	Categorical($\phi_{z_{dn}}$)	**Observed word** (shaded node). Chosen from the word distribution of the assigned topic.
ϕ_k	Dirichlet(β)	Word distribution for topic k. A V-dimensional probability vector.
β	fixed hyperparameter	Prior concentration on topic-word distributions. Controls sparsity of ϕ.

Generative process (LDA)

1. **For each topic** $k = 1, \ldots, K$:
   - $\phi_k \sim$ Dirichlet(β) (draw a word distribution for each topic)
2. **For each document** $d = 1, \ldots, D$:
   - $\theta_d \sim$ Dirichlet(α) (draw a topic mixture for the document)
3. **For each word** $n = 1, \ldots, N_d$:
   - $z_{dn} \sim$ Categorical(θ_d) (choose a topic)
   - $w_{dn} \sim$ Categorical($\phi_{z_{dn}}$) (choose a word from that topic)

What the plates mean

- **Inner plate (N_d):** repeat the topic and word selection for each of the N_d tokens in document d.
- **Outer plate (D):** repeat for each of the D documents in the corpus.
- **Separate plate (K):** repeat the topic word distribution for each of the K topics.
- **Shaded node:** w_{dn} is the only observed variable (the actual words in the documents). All other variables are latent and must be inferred.

Key points to remember

1. LDA assumes a bag-of-words model (word order ignored).
2. Each document is a mixture of topics (θ_d).
3. Each topic is a distribution over words (ϕ_k).
4. α controls how many topics a document tends to use (small $\alpha \rightarrow$ sparse, large $\alpha \rightarrow$ diffuse).
5. β controls how many words a topic tends to use (small $\beta \rightarrow$ specific words, large $\beta \rightarrow$ broad vocabulary).

The LDA generative process

The same story written as an algorithm

Generative algorithm

1. **For each topic k = 1 … K:**

   draw a word distribution $\varphi_k \sim \text{Dirichlet}(\beta)$

2. **For each document d = 1 … D:**

   draw topic proportions $\theta_d \sim \text{Dirichlet}(\alpha)$

   for each token n = 1 … N_d:

   draw a topic $z_{dn} \sim \text{Categorical}(\theta_d)$

   draw a word $w_{dn} \sim \text{Categorical}(\varphi_{z_{dn}})$

Assumptions baked in

- Bag of words — token order within a document is irrelevant.
- Exchangeable documents — corpus order is irrelevant.
- K is known and fixed in advance.
- Topics are shared globally across the corpus.

Why two Dirichlets

α makes each document use few topics; β makes each topic use few words. Together they produce the sparse, readable structure that makes LDA output interpretable.

Running this forward generates fake documents. Running it backward — given real documents, recover z, θ and ϕ — is the inference problem of Part 4.

The joint distribution and what must be inferred

Bayes' rule is easy to write and impossible to compute directly

$$p(w, z, \theta, \varphi \mid \alpha, \beta) = \Pi_k\, p(\varphi_k|\beta) \cdot \Pi_d\, p(\theta_d|\alpha) \cdot \Pi_n\, p(z_{dn}|\theta_d) \cdot p(w_{dn}|\varphi_{z_{dn}})$$

The joint — a direct transcription of the generative algorithm.

$$p(z, \theta, \varphi \mid w) = p(w, z, \theta, \varphi) \,/\, p(w)$$

The posterior we want.

Why it is intractable

The evidence p(w) requires summing over every possible topic assignment: K^N terms for N tokens. With K = 20 and N = 1,000 that is 20^{1000}.

Variational Bayes

Replace the posterior with a simpler factorised family and optimise. Deterministic, fast, scales online.

Collapsed Gibbs sampling

Integrate out θ and ϕ, then sample each z_{dn} in turn. Simple, accurate, the focus of Part 4.

Expectation Propagation

Moment-matching approximation. Accurate but rarely used in practice.

Micro worked example — three documents, two topics

Hand-initialised assignments on the toy corpus from Part 1

Doc	Tokens	Random initial assignment z	Doc-topic counts n(d,·)
d_1	patient, hospital, scan	T1, T2, T1	T1: 2 T2: 1
d_2	hospital, server, breach	T2, T1, T2	T1: 1 T2: 2
d_3	server, breach, attacker, data	T1, T2, T2, T1	T1: 2 T2: 2

Topic-word count table after initialisation

Word	n(T1, w)	n(T2, w)
patient	1	0
hospital	1	1
scan	1	0
server	2	0
breach	0	2
attacker	0	1
data	1	0

Where we are

The assignments are random, so the two topics are still meaningless — 'hospital' sits in both, and 'server' is split across documents.

Nothing yet separates care words from security words.

What happens next

Gibbs sampling revisits one token at a time and reassigns it using these very counts. After enough sweeps, care words concentrate in one topic and security words in the other.

We trace one such reassignment by hand in Part 4.

LDA Algorithm, Gibbs Sampling & Implementation

Deriving the collapsed Gibbs update, tracing it on real counts, then running it in code.

Gibbs sampling

A method that samples one variable at a time from its conditional

The general recipe

To sample from a joint $p(x_1,\ldots,x_m)$ that you cannot sample directly, repeatedly draw each variable from its conditional given all the others:

$$x_i \sim p(x_i \mid x_1,\ldots,x_{i-1}, x_{i+1},\ldots,x_m)$$

The chain converges to the true joint distribution.

Why "collapsed" for LDA

θ and ϕ are integrated out analytically using Dirichlet–Multinomial conjugacy (Part 2). Only the discrete topic assignments z remain to be sampled.

Fewer variables → lower variance and faster mixing than sampling z, θ and ϕ together.

Burn-in

Early samples reflect the random initialisation and are discarded — typically the first 100–500 sweeps.

Mixing

How quickly the chain forgets where it started. Sparse priors and good preprocessing help.

Thinning

Keep every m-th sample to reduce autocorrelation between retained draws.

Estimation

Average count tables over retained samples to estimate θ and ϕ.

Variational inference optimises an approximation; Gibbs sampling draws from the true posterior — given enough sweeps.

The collapsed Gibbs update equation

The single formula the whole algorithm runs on

$$p(z_i = k \mid z_{-i}, w) \propto (n_{-i}(d,k) + \alpha_k) \times (n_{-i}(k, w_i) + \beta) / (n_{-i}(k, \cdot) + V\beta)$$

Subscript −i means: remove token i from every count before evaluating.

$n_{-i}(d, k) + \alpha_k$

Document affinity

How much does document d already use topic k? Plus the prior pseudo-count α.

$n_{-i}(k, w_i) + \beta$

Word affinity

How often has topic k generated this specific word elsewhere in the corpus? Plus β.

$n_{-i}(k, \cdot) + V\beta$

Normaliser

Total tokens assigned to topic k, making the word term a proper probability.

Read it as a tug of war: a token is pulled towards topics its document already favours, and towards topics that already like that word. The product of the two pulls decides the draw.

The collapsed Gibbs sampling algorithm

Initialise, then sweep the corpus until the chain settles

Pseudocode

```
initialise: assign every token a random topic
update counts n(d,k), n(k,w), n(k,·)
for sweep = 1 .. S:
    for each document d:
        for each token i in d:
            k_old = z[i]
            decrement n(d,k_old), n(k_old,w_i), n(k_old,·)
            for k in 1..K:
                p[k] = (n(d,k)+a) * (n(k,w_i)+b) / (n(k,.)+V*b)
            k_new = sample from normalise(p)
            z[i] = k_new
            increment n(d,k_new), n(k_new,w_i), n(k_new,·)
after burn-in: accumulate counts for the estimates
```

$$\hat{\theta}(d,k) = (n(d,k) + \alpha_k) \,/\, (N_\partial + \Sigma\alpha_k)$$

$$\hat{\varphi}(k,w) = (n(k,w) + \beta) \,/\, (n(k,\cdot) + V\beta)$$

Read the estimates off the final count tables.

Cost and practicalities

- O(N·K) per sweep — linear in tokens and topics.
- Only integer count tables are stored — memory-light.
- Typically 500–2000 sweeps; monitor the log-likelihood.
- **Decrement-before-sample is the step people get wrong.**

Everything in the inner loop is count arithmetic — which is why a readable Python implementation fits on one slide.

Tracing one Gibbs step by hand

Resampling the token “server” in document d_3 ($\alpha = 0.5$, $\beta = 0.1$, $V = 7$)

1 · Remove the token

Token “server” in d_3 currently sits in T1. Decrement its three counts:

$$n_{-i}(d_3,T1) = 1,\ n_{-i}(d_3,T2) = 2$$
$$n_{-i}(T1,\text{server}) = 1,\ n_{-i}(T2,\text{server}) = 0$$
$$n_{-i}(T1,\cdot) = 4,\ n_{-i}(T2,\cdot) = 5$$

2 · Score every topic with the update equation

$$p(T1) \propto (1 + 0.5) \times (1 + 0.1) / (4 + 7{\times}0.1) = 1.5 \times 1.1 / 4.7 = 0.3511$$
$$p(T2) \propto (2 + 0.5) \times (0 + 0.1) / (5 + 7{\times}0.1) = 2.5 \times 0.1 / 5.7 = 0.0439$$

normalise: $p(T1) = 0.889$, $p(T2) = 0.111$

3 · Draw and update

Sample from (0.889, 0.111). Suppose T1 is drawn — re-increment $n(d_3,T1)$, $n(T1,\text{server})$ and $n(T1,\cdot)$, and move to the next token.

Why T1 dominates here

T1 has already generated “server” once elsewhere, while T2 never has. The word-affinity term (1.1 vs 0.1) overwhelms T2's slight document advantage.

It is a draw, not an argmax

T2 still has an 11% chance. That residual randomness is what lets the chain escape a poor initialisation instead of locking in the first guess.

Implementation

The sampler from scratch, and the production route with gensim

From scratch — the algorithm of the previous slide, verbatim

```
import numpy as np
def collapsed_gibbs(docs, V, K=2, alpha=0.5,
                    beta=0.1, sweeps=1000):
    ndk = np.zeros((len(docs), K))   # doc-topic
    nkw = np.zeros((K, V))           # topic-word
    nk  = np.zeros(K)                # topic total
    z = [np.random.randint(K, size=len(d)) for d in docs]
    for d, doc in enumerate(docs):        # initialise
        for i, w in enumerate(doc):
            k = z[d][i]
            ndk[d, k] += 1; nkw[k, w] += 1; nk[k] += 1
    for _ in range(sweeps):
        for d, doc in enumerate(docs):
            for i, w in enumerate(doc):
                k = z[d][i]
                ndk[d, k] -= 1; nkw[k, w] -= 1; nk[k] -= 1
                p = (ndk[d] + alpha) * (nkw[:, w] + beta) \
                    / (nk + V * beta)
                k = np.random.choice(K, p=p / p.sum())
                z[d][i] = k
                ndk[d, k] += 1; nkw[k, w] += 1; nk[k] += 1
    return ndk, nkw, z
```

In production — gensim

```
from gensim import corpora, models
dictionary = corpora.Dictionary(tokenised_docs)
dictionary.filter_extremes(no_below=5, no_above=0.5)
corpus = [dictionary.doc2bow(d) for d in tokenised_docs]
lda = models.LdaModel(corpus, id2word=dictionary,
        num_topics=20, alpha='auto', eta=0.01,
        passes=10, iterations=400, random_state=42)
lda.print_topics(num_words=10)
theta = lda.get_document_topics(corpus[0])
```

Parameter	Typical value	Effect
num_topics (K)	10 – 200	Granularity; tune with coherence
alpha	'auto' or 50/K	Document sparsity
eta (β)	0.01 – 0.1	Topic-word sparsity
passes / iterations	10 / 400	Convergence quality

Evaluating the model and choosing K

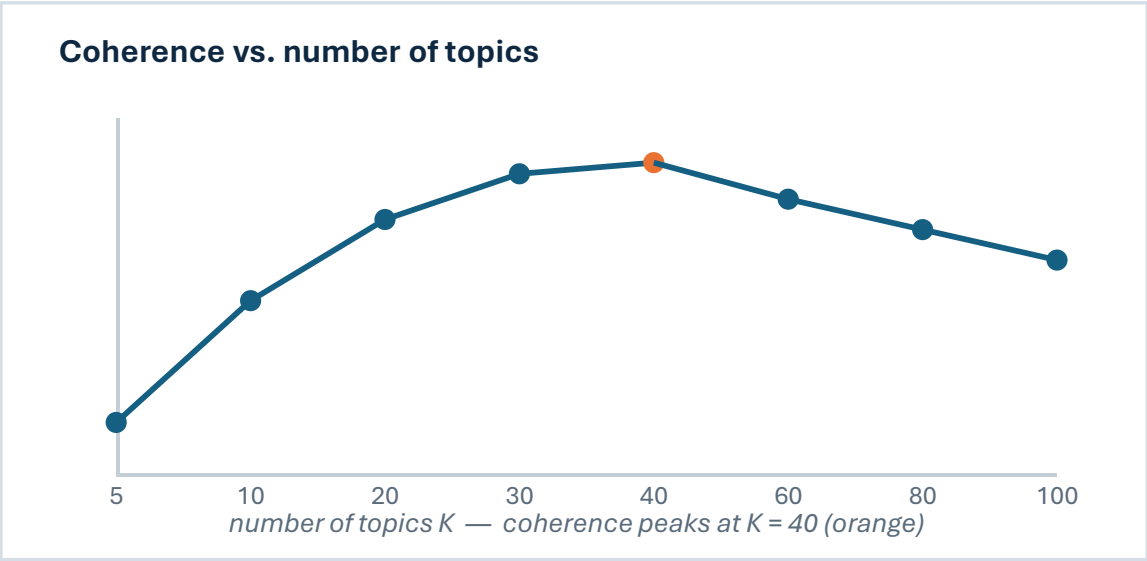
Coherence tracks human judgement far better than perplexity

Two numbers and one human

- Perplexity — held-out likelihood. Lower is better, but it often disagrees with what people find readable.
- Topic coherence (C_v, UMass) — scores how often the top words of a topic co-occur. Higher is better.
- **Inspection — read the top 10 words of every topic. Always.**

How to choose K in practice

Fit the model over a grid of K, plot coherence, and pick the peak — then sanity-check the neighbouring values by reading their topics. Prefer the smaller K when scores tie.



Warning signs: near-duplicate topics (K too large), topics mixing unrelated themes (K too small), and a topic of pure stopwords (preprocessing is too loose).

Pitfalls, extensions and key takeaways

What to remember, what to avoid, and where to go next

Pitfalls to avoid

- Trusting topic labels without reading the top words.
- Running LDA on tweets or search queries — documents are too short.
- Comparing topic indices across runs; they permute freely.
- Skipping burn-in and reading the first sample.

Beyond vanilla LDA

- Correlated Topic Models — topics that co-occur.
- Dynamic Topic Models — topics that evolve over time.
- Hierarchical Dirichlet Process — infers K automatically.
- BERTopic / Top2Vec — embedding-based, context aware.

1

Two mixtures

Documents mix topics; topics mix words.

2

Dirichlet = smoothing

Priors are pseudo-counts that keep everything non-zero.

3

Conjugacy = tractability

Integrating θ and ϕ out is what makes collapsed Gibbs simple.

4

Counts are the model

The whole sampler is arithmetic on three count tables.

Further reading: Blei, Ng & Jordan (2003), “Latent Dirichlet Allocation”, JMLR · Griffiths & Steyvers (2004), “Finding Scientific Topics”, PNAS · gensim and scikit-learn documentation.